Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 00 (2025) 000–000

Procedia Computer Science

www.elsevier.com/locate/procedia

International Conference on Machine Learning and Data Engineering

Real-Time Adaptive Cross-Media Recommendation Using Semantic Embeddings, Knowledge Graph Scoring, and Graph Collaborative Learning

Sanjay A. K.[a], Eswara Raj M.[a], Aditya Vijay[a], Himaneesh Reddy Jangalapalli[a], Bagavathi Sivakumar P.[a,*], Krishna Priya G.[a,*]

[a]*Department of Computer Science and Engineering, Amrita School of Computing, Coimbatore, Amrita Vishwa Vidyapeetham, India*

Abstract

A user's interests span media: the same preference can appear as a film theme, a book topic, a musical artist, or a learning goal, yet most recommenders operate within single-domain silos. This paper presents a real-time adaptive cross-media recommender that maps movies, TV shows, books, and music, extended to health, industrial, and finance products, into one catalog of 120,747 items. Candidates from Sentence-BERT embeddings with Qdrant nearest-neighbor search and from creator and title matching are reranked by a hybrid score combining exponential moving average (EMA) session adaptation, LightGCN collaborative filtering, and knowledge-graph (KG) and profile scoring, behind a FastAPI and React application with MySQL interaction logging. On three Amazon Reviews 2023 categories with up to 50,000 users, the hybrid with validation-selected weights is the best method on two categories, improving NDCG@20 over the strongest single signal by 21.6% and 26.9%, and an ablation shows that every signal contributes significantly; on the third category, entity overlap alone performs best. A recommendation over the full catalog takes 363 ms at the median on a laptop, and a feedback event updates the user profile in 0.05 ms.

© 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering.

Keywords: Recommender systems; Cross-media recommendation; Sentence-BERT; Qdrant; LightGCN; Semantic search; Graph collaborative filtering; Real-time personalization; Knowledge graph

1. Introduction

Recommender systems are the main interface through which users discover digital content, yet most deployed systems remain domain-specific: a movie platform recommends films, a music platform recommends tracks, and a book platform recommends books. Human interest does not follow these boundaries. A user interested in science

* Corresponding authors.
E-mail address: g_krishnapriya@cb.amrita.edu (Krishna Priya G.), pbsk@cb.amrita.edu (Bagavathi Sivakumar P.)

1877-0509 © 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering.

fiction may enjoy a film such as *Interstellar*, a novel about space exploration, a soundtrack with a similar tone, and educational content on astronomy, so a useful recommender should connect these signals across media.

This paper presents a working cross-media recommendation pipeline for movies, TV shows, books, and music, which has since been extended to health, industrial, and finance products. Every item is mapped to one schema and one semantic retrieval layer: Sentence-BERT compares items of different domains through their descriptions [1], Qdrant provides nearest-neighbor retrieval [2], LightGCN learns collaborative preference [3], an exponential moving average (EMA) of item embeddings adapts to each feedback event, and an in-memory knowledge graph (KG) adds entity- and profile-aware scoring, behind a FastAPI backend [4] with MySQL logging.

These capabilities are rarely combined in one multi-domain pipeline: graph models are usually benchmarked on single-domain datasets, semantic systems retrieve by meaning rather than by preference, cross-domain methods typically transfer between two domains instead of normalizing many into one catalog, and real-time personalization is usually separate from graph learning. The contributions of this paper are threefold:

1. An integrated cross-media platform with a unified catalog of seven content types that supports semantic, creator-aware, and title search.
2. A hybrid ranking model that combines semantic retrieval, EMA adaptation, LightGCN graph scoring, and KG/profile scoring, with explainable sub-scores.
3. A quantitative evaluation on three public interaction datasets, comprising single-signal baselines, an ablation of every signal, validation-based weight selection, latency measurements, and scenario tests of the deployed system.

2. Related Work and Novelty

Collaborative filtering learns from behavior and content-based recommendation from item metadata, and hybrid approaches combine the two to reduce the cold-start problem [5]. NGCF applies graph convolution to the user–item graph, and LightGCN simplifies it by keeping only neighborhood propagation [6, 3]; here LightGCN is a reranking signal for users with interaction history rather than the candidate generator. Sequential recommenders such as SASRec model the order of user actions within a single domain [7], and Sentence-BERT embeddings [1] let items of different media share one vector space before any interactions exist. KGAT and RippleNet propagate preferences over knowledge-graph relations such as directed-by [8, 9]. Cross-domain recommendation transfers preferences between domains, mostly between two [10], and a recent survey extends the scope to multi-domain transfer [11]; large language models augmented with graph structure have also been used for relevance and explanation [12]. Studies from Amrita Vishwa Vidyapeetham have addressed cold-start prediction, collaborative filtering, sparse data, dual-feedback personalization, and community-aware recommendation [13, 14, 15, 16, 17], but not the integration of heterogeneous media into one pipeline.

Recent work has moved in two directions that are directly relevant here. Knowledge-aware graph recommenders combine LightGCN-style propagation with knowledge graphs: LGKAT adds a personalized knowledge-aware attention sub-network to LightGCN and is evaluated separately on movie, book, music, and restaurant data [18], and LightKG simplifies the graph neural network of knowledge-aware recommenders to reduce training cost on sparse data [19]. Cross-domain recommenders increasingly use large language models as a semantic bridge between domains: LLMCDSR generates pseudo-interactions with an LLM and injects collaborative information into textual item features [20], LLM4CDSR builds unified LLM-based item representations and hierarchical user profiles across domains [21], LeCDSR likewise enhances cross-domain sequential recommendation with an LLM [22], and KLLMRec augments multi-domain recommendation with LLM-derived knowledge [23].

Table 1 contrasts these approaches, based on the components reported in each cited paper. Knowledge-aware graph models learn from one domain at a time, and LLM-based cross-domain models are trained offline for a fixed set of domains around next-item prediction; neither family combines text-based retrieval, collaborative graph scoring, KG/profile scoring, and per-event adaptation in a deployed system. The novelty of this work is therefore not a new neural layer but the integration of these methods over one catalog, with immediate (EMA) and periodic (LightGCN) personalization kept separate; in exchange, its knowledge graph is a lightweight entity-overlap scorer rather than a learned embedding, and its encoder is a compact sentence model rather than an LLM. Because the cited models

Table 1. Comparison with representative graph, knowledge-aware, and cross-domain recommenders.

Method	Domains	Text semantics	Collab. graph	Knowledge graph	Short-term adapt.	Deployed
LightGCN [3]	single	–	yes	–	–	–
KGAT [8]	single	–	yes	yes	–	–
LGKAT [18]	single	–	yes	yes	–	–
LightKG [19]	single	–	yes	yes	–	–
LLMCDSR [20]	cross-domain	LLM	–	–	sequence	–
LLM4CDSR [21]	cross-domain	LLM	–	–	sequence	–
This work	7 types, one catalog	Sentence-BERT	LightGCN	entity overlap	per-event EMA	yes

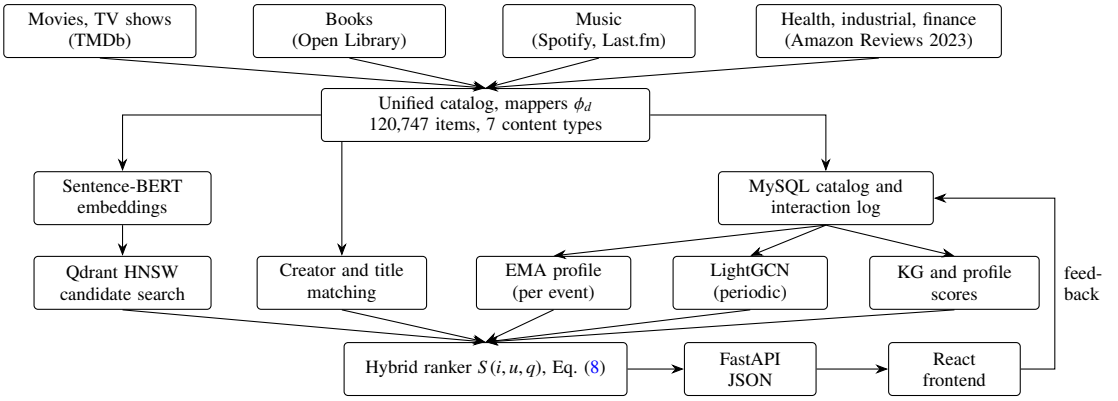


Fig. 1. System architecture of the proposed cross-media recommender.

address different tasks and training protocols, Section 5 does not reimplement them but isolates the contribution of each signal of the proposed pipeline under one protocol.

3. Implementation

3.1. System overview

In the architecture of Fig. 1, each source is mapped into the unified catalog, embedded, and loaded into Qdrant and MySQL offline, and LightGCN is retrained periodically. At request time, the FastAPI backend validates a query together with an optional user, content-type or domain filter, viewer age, and result count K; it matches creators and titles, retrieves Qdrant candidates, reranks them, and returns the final score with every sub-score, so that the React frontend can show which signal drove each recommendation. Feedback such as likes, skips, and bookmarks is logged through the interaction endpoint and updates the user's EMA profile immediately. The declared viewer age and domain act as a hard eligibility filter before ranking: an undeclared viewer is capped at a teen ceiling, and the health, industrial, and finance catalogs default to adult maturity.

3.2. Unified catalog and data sources

Each domain $d \in \mathcal{D}$ provides a mapper ϕ_d from a raw row x to the shared item representation r_i, and the catalog is

$$C = \bigcup_{d \in \mathcal{D}} \{\phi_d(x) : x \in X_d\}, \quad r_i = (id, type, title, desc, creators, categories, text_i). \tag{1}$$

Adding a domain therefore requires only a new ϕ_d; embedding, search, KG scoring, the API, and the frontend are unchanged. Movies and TV shows use TMDb metadata and credits [24]; books use the Open Library APIs [25]; music combines a public Spotify songs export with current tracks from the Last.fm API [26]; and health, industrial, and finance products come from the Amazon Reviews 2023 metadata [27], with finance also including accounting and personal-finance titles from Open Library. The deployed catalog holds 120,747 items: 31,133 songs, 23,492 movies, 20,000 health and 20,000 industrial products, 12,402 books, 10,617 TV shows, and 3,103 finance items. Directors and cast, authors, and artists all map to the shared creators field, which therefore serves both person search and the knowledge graph. A SHA-256 hash of each embedding text lets unchanged rows reuse their embeddings.

3.3. Semantic retrieval and creator and title matching

Each item's embedding text (title, categories, description, and creators) and each query are encoded by the all-MiniLM-L6-v2 Sentence-BERT model into 384-dimensional vectors [1]. The users' EMA profiles (Section 3.4) live in the same space, so users, items, and queries are compared directly. With $\mathbf{e}_i$ the embedding of item i and $\mathbf{q}$ that of the query, the semantic score is the cosine similarity $S_{sem}(i,q) = \mathbf{e}_i^\top \mathbf{q}/(\|\mathbf{e}_i\|\,\|\mathbf{q}\|)$. Qdrant performs approximate nearest-neighbor search over this similarity with an HNSW index; it generates candidates rather than the final ranking and lets a new item be recommended from its metadata alone.

Dense similarity does not capture the intent of a name or a short title such as *F1*, so two deterministic stages run first. Creator matching looks for exact, prefix, and substring matches on normalized creator names and places up to five matching items first. Title matching assigns each item one of five tiers on whole normalized words: the query equals the title (tier 0), the title starts with the query (1) or contains it (2), every query token occurs in the title or categories (3), or the query phrase occurs in the description (4). Tiers 0 and 1 rank first and tiers 2–4 are ordered by popularity; lexical matches fill at most $\max(3, \lfloor K/2 \rfloor)$ slots and receive the fixed score $1.30 - 0.06\,t$ for tier t, which keeps them above the cosine band. Semantic candidates fill the remaining slots after deduplication.

3.4. Interaction logging and EMA adaptation

Interactions are logged in MySQL with a signed weight w: view 0.4, click 0.6, like and bookmark 1.0, complete 1.2, and skip -1.0; an explicit rating $r \in [1,5]$ maps to $w = (r-3)/2$, and events with $w = 0$ are ignored. Let $\mathbf{u}_t \in \mathbb{R}^{384}$ be the user's profile and $\alpha = 0.25$ the smoothing factor. An event with weight w on item i moves the profile by the step $\eta = \alpha \min(|w|, 1)$ toward $\operatorname{sign}(w)\,\mathbf{e}_i$:

$$\tilde{\mathbf{u}}_{t+1} = (1-\eta)\,\mathbf{u}_t + \eta \operatorname{sign}(w)\,\mathbf{e}_i, \qquad \mathbf{u}_{t+1} = \tilde{\mathbf{u}}_{t+1}/\|\tilde{\mathbf{u}}_{t+1}\|. \tag{2}$$

The profile is empty until the user's first non-neutral event on an embedded item, which sets $\mathbf{u}_1 = \operatorname{sign}(w)\,\mathbf{e}_i$, so users without events receive no EMA score. Likes pull the profile toward an item, skips and low ratings push it away, weak signals such as views move it less, and renormalization keeps it on the unit sphere. The EMA score is the cosine similarity $S_{ema}(i,u) = \mathbf{e}_i^\top \mathbf{u}_t/(\|\mathbf{e}_i\|\,\|\mathbf{u}_t\|)$. Because each event updates a single vector, EMA can follow session-level drift without retraining any model.

3.5. Graph collaborative filtering with LightGCN

LightGCN [3] is trained on the interaction log. An event with $w > 0$ becomes an unweighted edge of the bipartite graph $\mathcal{G} = (\mathcal{U} \cup \mathcal{I}, \mathcal{E})$, repeated events on the same pair are merged, and skips and low ratings are excluded rather than used as explicit negatives. With $\mathcal{N}_u$ the neighbors of user u, layer ℓ propagates and the final embedding averages all L layers,

$$\mathbf{e}_u^{(\ell)} = \sum_{i \in \mathcal{N}_u} \frac{\mathbf{e}_i^{(\ell-1)}}{\sqrt{|\mathcal{N}_u|\,|\mathcal{N}_i|}}, \qquad \mathbf{e}_u = \frac{1}{L+1} \sum_{\ell=0}^{L} \mathbf{e}_u^{(\ell)}, \tag{3}$$

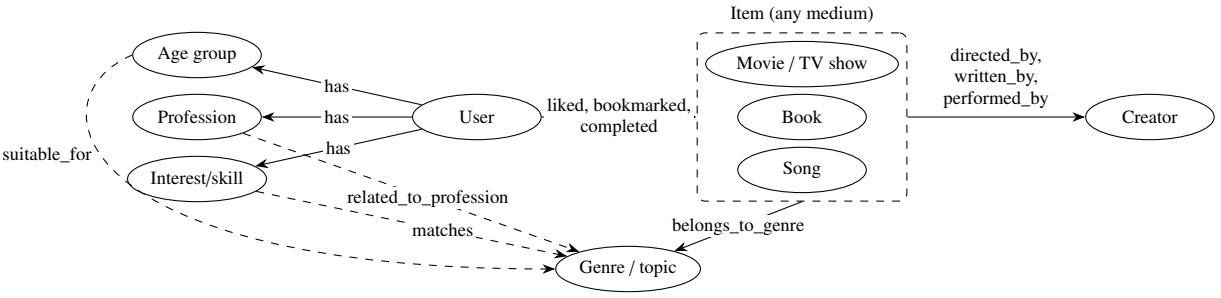


Fig. 2. Knowledge-graph schema; dashed edges are profile expansions into topic terms.

and symmetrically for items. Training minimizes the Bayesian Personalized Ranking (BPR) loss

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i,j)\in\mathcal{D}} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda\|\Theta\|^2, \tag{4}$$

where $\hat{y}_{ui} = \mathbf{e}_u^\top \mathbf{e}_i$, Θ are the layer-0 embeddings of the batch, and for every sampled positive pair (u, i) one negative j is drawn uniformly from the catalog and redrawn if (u, j) is observed. BPR suits implicit, positive-only feedback because it optimizes the pairwise ranking order directly, without negative labels [3, 6]. At inference time the graph score is the cosine similarity of the ℓ_2-normalized embeddings, $S_{graph}(i, u) = \mathbf{e}_u^\top \mathbf{e}_i / (\|\mathbf{e}_u\| \, \|\mathbf{e}_i\|)$. A graph score exists only when both user and item are in the training graph; otherwise the candidate is ranked by the other signals. The training settings are given in Section 4.

3.6. Knowledge graph and profile-aware scoring

The knowledge graph is built in memory from the catalog, without a separate graph database, in three steps (Fig. 2). *(i) Entities.* Each item becomes a node typed by its content type. After Unicode normalization and case-folding, each creator becomes a Creator node, each category a Genre/topic node, and each alphanumeric token of at least three characters in the title, description, creators, categories, and content type a term entity of the item. Each user becomes a User node, and the declared profile attributes become Profession, Age group, and Interest/skill nodes. *(ii) Relations.* Items link to creators by directed_by, written_by, or performed_by, according to the content type, and to genres by belongs_to_genre; users link to items by liked, bookmarked, or completed when the event weight is positive (these edges also form the LightGCN graph) and to their profile nodes. Each profession and age group expands into a fixed list of related topic terms (related_to_profession and suitable_for; e.g., *data analyst* → data, analytics, business, statistics, technology, science), and an age group may also carry unsuitable terms (e.g., *family* → horror, crime, thriller, war). *(iii) Paths.* A user reaches an item through a length-two path via a shared entity, and because entity namespaces are shared by all domains, the path can end at an item of any medium:

$$P(u, i) = \{u \to e \to i : \; e \in \mathcal{E}_{query} \cup \mathcal{E}_{creator} \cup \mathcal{E}_{genre} \cup \mathcal{E}_{profile}\}. \tag{5}$$

The two scores below count such paths through set intersections, without enumerating them.

The KG score counts paths through query-term entities. With $\mathcal{T}(q)$ and $\mathcal{T}(i)$ the term sets of query q and item i,

$$S_{kg}(i, q) = \min\left(1, \frac{|\mathcal{T}(q) \cap \mathcal{T}(i)|}{\max(3, |\mathcal{T}(q)|)}\right), \tag{6}$$

where the floor of three prevents a one- or two-word query from reaching the maximum through a single shared term. The profile score rewards items that match the user's declared attributes and penalizes unsuitable ones:

$$S_{profile}(i,u) = \sum_{t \in \mathcal{P}_+(u)} \delta(t,i) w_+ - \sum_{t \in \mathcal{P}_-(u)} \delta(t,i) w_- + \mathbf{1}[\text{pref}(u) = \text{type}(i)] w_c, \tag{7}$$

where $\mathcal{P}_+(u)$ holds the user's interests, skills, and the terms expanded from the profession and age group, $\mathcal{P}_-(u)$ the unsuitable terms, $\delta(t,i) = 1$ if term t occurs in the text of item i, $\text{pref}(u)$ the preferred content types, $w_+ = 0.16$, $w_- = 0.25$, and $w_c = 0.15$; the score is clipped to $[-1, 1]$. The profile uses only declared, consent-based attributes, not private social-media data.

3.7. Hybrid ranking

Candidates are sorted by the hybrid score

$$S(i,u,q) = S_{sem}(i,q) + B_{pref}(i,u) + \lambda_g S_{graph}(i,u) + \lambda_e S_{ema}(i,u) + \lambda_k S_{kg}(i,q) + \lambda_p S_{profile}(i,u), \tag{8}$$

where S_{sem} is replaced by the fixed lexical score for creator and title matches. B_{pref} is a short-term preference bonus from the user's 500 most recent events: with $c_u(g)$ and $c_u(\tau)$ the summed event weights of category g and content type τ,

$$B_{pref}(i,u) = 0.035 \sum_{g \in \text{cat}(i)} \min(c_u(g), 3) + 0.02 \min(c_u(\text{type}(i)), 3) - 0.15\, \mathbf{1}[i \in \mathcal{I}_u^+] - 0.35\, \mathbf{1}[i \in \mathcal{I}_u^-], \tag{9}$$

where $\mathcal{I}_u^+$ and $\mathcal{I}_u^-$ are the items the user has already rated positively and negatively, which are demoted so that new items are favored. For known users, $5K$ candidates are retrieved before reranking, so that personalization can promote items from outside the initial top K. The deployed system uses the hand-set weights $\lambda_g = 0.20$, $\lambda_e = 0.15$, $\lambda_k = 0.08$, and $\lambda_p = 0.10$; Section 5 compares them with weights selected on a validation split.

4. Experimental Setup

The system is evaluated with an offline benchmark on public interaction data, which measures every signal of Eq. (8) under identical conditions, and with scenario tests of the deployed system for the query- and profile-dependent behavior that public logs cannot exercise.

Datasets. The benchmark uses three categories of the Amazon Reviews 2023 corpus [27]: Movies & TV, Health & Household, and Industrial & Scientific. Ratings of four or five stars are positive interactions; other ratings are discarded. The two large categories are filtered to a 10-core graph and a random sample of 50,000 users is drawn, after which items with fewer than five remaining interactions are removed; Industrial & Scientific does not survive a 10-core filter and uses the 5-core graph without sampling. Item text (title, category path, description, and store) comes from the corpus metadata. Table 2 lists the graphs, whose densities are about 0.04%; metadata covers every Health & Household and Industrial & Scientific item but only 67.3% of the Movies & TV items, and items without text receive no semantic, EMA, or KG signal.

Split, candidates, and metrics. Each user's positives are ordered by timestamp; the last two form the test set, the one before them the validation set, and the rest the training set, on which alone LightGCN is trained. Every method ranks the full training catalog minus the user's history, so evaluation involves no negative sampling. Recall@K, NDCG@K, MAP@K, and MRR@K are averaged over test users for $K \in \{10, 20\}$, and significance is assessed with a paired Wilcoxon signed-rank test on per-user NDCG@20.

Table 2. Benchmark graphs after filtering.

Category	Users	Items	Training positives	Test users	Items with text
Movies & TV	50,000	52,213	814,346	49,994	35,153 (67.3%)
Health & Household	50,000	42,621	709,212	50,000	42,621 (100.0%)
Industrial & Scientific	33,959	18,804	170,834	33,926	18,804 (100.0%)

Table 3. LightGCN and EMA settings.

Setting	Value	Setting	Value
Embedding size / layers	64 / 3, averaged (Eq. (3))	Negative sampling	1 uniform per positive, resampled if observed
Initialization	$\mathcal{N}(0, 0.1^2)$	Positive interactions	$w > 0$ (Amazon: rating ≥ 4), merged
Optimizer / lr / L_2 λ	Adam / 10^{-3} / 10^{-4}	EMA α / step	0.25 / $\eta = \alpha \min(\lvert w \rvert, 1)$
Batch / epochs	4096 / 60, no early stopping	Text encoder	all-MiniLM-L6-v2, 384-d

Offline signals. Public logs contain neither typed queries nor declared profiles, so each signal uses its closest offline counterpart: S_{sem} takes the user's most recent history item as the query, which corresponds to the system's similar-item endpoint; S_{ema} replays Eq. (2) over the history in time order; S_{graph} is the LightGCN cosine score; and S_{kg} applies Eq. (6) to the entities of the user's history (category-path nodes, store, and main category), normalized by the candidate's entity count because a history holds far more entities than a query. Entities occurring on a single item or on more than half of the items are dropped. The declared-profile score has no offline counterpart and is covered by the scenarios, and B_{pref} is omitted because its category term overlaps with the offline KG signal.

Baselines and weight selection. The baselines are most-popular ranking and each signal on its own. The hybrid is evaluated with the hand-set weights and with weights chosen by grid search on 10,000 validation users, with a semantic weight $\lambda_s \in \{0, 0.5, 1\}$ (fixed to 1 in Eq. (8)) and $\lambda_g, \lambda_e, \lambda_k \in \{0, 0.05, 0.1, 0.2, 0.4, 0.8, 1.6\}$, maximizing validation NDCG@20; the test set is used once, for the selected weights. The ablation removes one signal at a time from the selected hybrid.

Training settings. Table 3 lists the settings. LightGCN is trained for a fixed 60 epochs of $\lceil |\mathcal{E}|/B \rceil$ mini-batches without early stopping, so that neither validation nor test data influence training. All runs use seed 42 on one NVIDIA T4 GPU, on which a training run takes 17.1, 13.2, and 1.2 minutes on the three graphs.

5. Results

5.1. Baselines and ablation

Table 4 reports the results at $K = 20$. Absolute values are low, as usual for full-catalog ranking on sparse purchase data, so methods are compared under identical conditions. Every difference between the tuned hybrid and another row is significant ($p < 0.01$), and the results at $K = 10$ lead to the same conclusions, except that the tuned and hand-set hybrids are tied on Health & Household.

On Movies & TV and Industrial & Scientific, the hybrid with validation-selected weights is the best method on all four measures at both cut-offs. It improves NDCG@20 over the strongest single signal, semantic similarity, by 21.6% and 26.9%, and over most-popular ranking by factors of 2.3 and 1.9. The signals are complementary: LightGCN alone is weaker than semantic retrieval on every dataset, yet removing it from the hybrid lowers NDCG@20 by 14.6%, 21.8%, and 19.6% on the three datasets, almost as much as removing the semantic score (31.6%, 19.2%, and 21.3%). EMA and KG add smaller but consistent gains; removing either lowers NDCG@20 by 0.7–3.0% on every dataset.

Health & Household behaves differently. Semantic similarity, EMA, and LightGCN on their own all rank below most-popular, whereas KG overlap alone, which here matches category paths and brands, is the best method and exceeds the tuned hybrid by 14.4% in NDCG@20. Repeat purchases within a brand or category, which the entity-overlap score captures directly, dominate this domain. The validation set nevertheless selected a small KG weight, because one held-out item per user gives a noisy estimate of the best weights; the hybrid still beats most-popular

Table 4. Offline ranking results at $K = 20$ (R: Recall, N: NDCG); the best value in each column is in bold.

Method	Movies & TV				Health & Household				Industrial & Scientific			
	R@20	N@20	MAP@20	MRR@20	R@20	N@20	MAP@20	MRR@20	R@20	N@20	MAP@20	MRR@20
Most popular	0.0197	0.0097	0.0048	0.0089	0.0237	0.0115	0.0056	0.0106	0.0282	0.0147	0.0077	0.0150
Semantic only	0.0307	0.0187	0.0118	0.0196	0.0166	0.0096	0.0054	0.0104	0.0379	0.0223	0.0134	0.0238
EMA only	0.0233	0.0128	0.0074	0.0126	0.0104	0.0052	0.0026	0.0050	0.0254	0.0135	0.0074	0.0131
LightGCN only	0.0257	0.0136	0.0075	0.0134	0.0116	0.0054	0.0025	0.0047	0.0242	0.0130	0.0072	0.0130
KG only	0.0093	0.0045	0.0021	0.0041	**0.0253**	**0.0144**	**0.0081**	**0.0157**	0.0364	0.0210	0.0122	0.0224
Hybrid, hand-set	0.0344	0.0216	0.0140	0.0230	0.0208	0.0122	0.0070	0.0133	0.0455	0.0266	0.0159	0.0282
Hybrid, tuned	**0.0368**	**0.0227**	**0.0146**	**0.0243**	0.0223	0.0126	0.0071	0.0135	**0.0495**	**0.0283**	**0.0167**	**0.0295**
− semantic	0.0290	0.0156	0.0088	0.0152	0.0206	0.0102	0.0051	0.0096	0.0409	0.0223	0.0126	0.0224
− EMA	0.0366	0.0226	0.0145	0.0241	0.0217	0.0123	0.0070	0.0133	0.0484	0.0275	0.0161	0.0286
− LightGCN	0.0314	0.0194	0.0124	0.0206	0.0171	0.0098	0.0056	0.0106	0.0389	0.0227	0.0136	0.0241
− KG	0.0362	0.0224	0.0144	0.0240	0.0215	0.0122	0.0069	0.0131	0.0485	0.0276	0.0162	0.0286

Table 5. Latency of *POST /recommend* by request type and stage, in milliseconds.

Request or stage	Mean	p50	p95	p99
End-to-end, anonymous ($n = 100$)	345.9	342.1	426.6	488.8
End-to-end, personalized ($n = 600$)	375.5	369.8	458.8	508.4
Creator matching	46.2	44.4	63.1	74.1
Lexical title/keyword matching	198.9	190.8	265.7	307.2
Query encoding (Sentence-BERT)	12.6	12.0	14.0	23.4
Qdrant HNSW search with payloads	85.2	82.6	106.3	126.4
MySQL preference bonus (B_{pref})	6.4	7.2	8.5	9.5
LightGCN rerank	0.07	0.07	0.13	0.16
EMA rerank	6.5	7.3	9.3	10.6
KG/profile rerank	13.1	13.1	22.3	28.2

ranking and every other single signal, but the result shows that weights should be tuned per domain on more validation data.

The selected weights are stable: relative to the semantic score, $\lambda_g = 0.8$, $\lambda_e = 0.2$–0.4, and $\lambda_k = 0.05$–0.1 (the selection $(0.5, 0.4, 0.2, 0.05)$ has the same ratios as $(1, 0.8, 0.4, 0.1)$). The hand-set weights thus under-weight the graph signal by a factor of four, and the tuned weights improve NDCG@20 over them by 5.4%, 3.6%, and 6.5% ($p < 10^{-4}$ on every dataset).

5.2. Latency

Latency was measured on a laptop (AMD Ryzen 5 5600H, 16 GB RAM, NVIDIA RTX 3050 Laptop GPU for query encoding) running MySQL 8.0 and a Qdrant 1.12.6 server with an HNSW index ($m = 16$, ef_construct = 100) over the full 120,747-item catalog. After 30 warm-up requests, 700 requests with $K = 10$ (16 queries, each issued anonymously and for six demo users) were sent through FastAPI's in-process test client, so validation, ranking, and serialization are included and only the network hop is excluded (Table 5).

Over all 700 requests, the median latency is 363 ms (mean 371 ms, p95 455 ms, p99 507 ms). Personalized requests are 28 ms slower at the median than anonymous ones: the MySQL preference, LightGCN, and EMA stages add about 15 ms, and the rest comes from retrieving $5K$ instead of K candidates. An EMA update takes 0.05 ms (p99 0.08 ms over 2,000 updates), so a like or skip is reflected in the very next request without retraining. The dominant costs are the lexical title scan (191 ms), which scans the catalog with pandas on every request, and the Qdrant query (83 ms), which returns up to $5K$ candidates with their payloads. Because the CPU-bound scan is serialized by the Python interpreter lock, one API process sustains 2.7 requests per second, and 3.3 with eight concurrent workers; an inverted title index

Table 6. Prototype scenarios run through the deployed recommendation API.

Scenario (demo user)	Age / domain	PersonHit@5	ProfileAlign@5	Avg. KG	Avg. profile	Content types in top 10
James Cameron (sci-fi user)	20	4/5	5/5	0.667	0.980	finance, movie, show
Taylor Swift (pop user)	20	5/5	5/5	0.667	0.630	book, movie, music
Business analytics (learning user)	30	–	5/5	0.650	1.000	book, finance
Fun animation adventure (family)	8	–	5/5	0.667	1.000	movie
Space adventure with aliens (sci-fi)	20	–	5/5	0.600	0.896	movie, show
Joint pain supplements (caregiver)	30 / health	–	5/5	0.900	1.000	health
Lab safety gloves (engineer)	30 / industry	–	5/5	0.767	0.694	industrial
Personal budgeting (finance user)	30 / finance	–	5/5	0.700	0.938	finance
Mean		0.90	1.00	0.702	0.892	4/8 cross-domain

and several worker processes are the main routes to higher throughput. Qdrant's embedded local mode, used during development, performs exact search in Python and took 1.5 s per query in the same setting, so the server deployment is required for interactive use.

5.3. Prototype scenarios

Eight scenarios were issued through the deployed endpoint with $K = 10$, each as a seeded demo user with the viewer age implied by its profile (20 for young adults, 30 for adults, 8 for the family profile); the three health, industry, and finance scenarios also declare their domain, as the frontend does. PersonHit@5 counts the top-5 results that credit the queried person, ProfileAlign@5 the top-5 results with a positive profile score (Eq. (7)), Avg. KG and Avg. profile are the mean S_{kg} and $S_{profile}$ of the top five, and the cross-domain rate is the fraction of scenarios whose top ten contains at least two content types.

Both person queries place four or five of the person's items in the top five; the James Cameron query ranks the series *James Cameron's Story of Science Fiction* first, which matches by title but does not list him as a creator. Every scenario places five profile-aligned items in the top five. Four of the five scenarios without a domain filter return at least two content types; the family profile receives only age-appropriate films; and the three domain-scoped scenarios stay in their declared domain by design. The cross-domain rate is thus $4/8$ overall and $4/5$ without a domain filter ($3/8$ and $3/5$ if movies and TV shows count as one medium). EMA scores were available for all 80 returned items but a LightGCN score for only one, because the deployed LightGCN model is trained on the demo log (13 users and 347 items) and rarely covers the candidates of a new query; its contribution is measured by the benchmark instead.

6. Conclusion and Future Work

This paper presented a real-time adaptive cross-media recommender that ranks seven content types from one catalog by combining semantic, creator and title, EMA, LightGCN, and KG and profile signals. On three Amazon Reviews 2023 categories, the hybrid with validation-selected weights was the best method on two of them, every signal contributed significantly, and weight selection improved on the hand-set weights; a request takes 363 ms at the median and a feedback update 0.05 ms. Its limitations are the Health & Household result, where entity overlap alone was best, the demo-only coverage of the deployed LightGCN model, and a throughput of about three requests per second; future work will address them with per-domain weight tuning and an inverted title index, and will add further domains and user studies.

References

[1] Reimers N, Gurevych I. Sentence-BERT: sentence embeddings using Siamese BERT-networks. In: Proceedings of EMNLP-IJCNLP; 2019. p. 3982–3992.

[2] Qdrant. Qdrant vector database documentation; 2026. Available from: https://qdrant.tech/documentation/

[3] He X, Deng K, Wang X, Li Y, Zhang Y, Wang M. LightGCN: simplifying and powering graph convolution network for recommendation. In: Proceedings of the 43rd ACM SIGIR Conference; 2020. p. 639–648.
[4] Ramírez S. FastAPI documentation; 2026. Available from: https://fastapi.tiangolo.com/
[5] Zhang S, Yao L, Sun A, Tay Y. Deep learning based recommender system: a survey and new perspectives. ACM Computing Surveys 2019;**52**(1):1–38.
[6] Wang X, He X, Wang M, Feng F, Chua TS. Neural graph collaborative filtering. In: Proceedings of the 42nd ACM SIGIR Conference; 2019. p. 165–174.
[7] Kang WC, McAuley J. Self-attentive sequential recommendation. In: Proceedings of the IEEE ICDM; 2018. p. 197–206.
[8] Wang X, He X, Cao Y, Liu M, Chua TS. KGAT: knowledge graph attention network for recommendation. In: Proceedings of the 25th ACM SIGKDD Conference; 2019. p. 950–958.
[9] Wang H, Zhang F, Wang J, Zhao M, Li W, Xie X, Guo M. RippleNet: propagating user preferences on the knowledge graph for recommender systems. In: Proceedings of the 27th ACM CIKM; 2018. p. 417–426.
[10] Zang T, Zhu Y, Liu H, Zhang R, Yu J. A survey on cross-domain recommendation: taxonomies, methods, and future directions. ACM Transactions on Information Systems 2022;**41**(2).
[11] Zhang H, Cheng M, Liu Q, Jiang J, Wang X, Zhang R, Lei C, Chen E. A comprehensive survey on cross-domain recommendation: taxonomy, progress, and prospects. arXiv preprint arXiv:2503.14110; 2025.
[12] Wei W, Ren X, Tang J, Wang Q, Su L, Cheng S, Wang J, Yin D, Huang C. LLMRec: large language models with graph augmentation for recommendation. In: Proceedings of the 17th ACM WSDM; 2024. p. 806–815.
[13] Lakshmi Chetana V, Batchu RK, Devarasetty P, Voddelli S, Prasad DV. Effective movie recommendation based on improved densenet model. Multiagent and Grid Systems 2023;**19**(2):133–147.
[14] Lakshmi Chetana V, Seetha H. Deep collaborative filtering with AdaMVRGO for movie recommendations. Traitement du Signal 2023;**40**(6).
[15] Lakshmi Chetana V, Seetha H. Handling massive sparse data in recommendation systems. Journal of Information and Knowledge Management 2024;**23**(3).
[16] Lakshmi Chetana V, Seetha H. A deep learning framework for enhancing recommender systems with dual-feedback integration. Applied Computational Intelligence and Soft Computing 2025.
[17] Keerthika T, Ignisha RG, Aiyyappan MR, Thoshi Babu LS, et al. Enhancing recommendation systems through SVD-based collaborative filtering and community detection. PLOS ONE 2026.
[18] Hassanzadeh R, Majidnezhad V, Arasteh B. A novel recommender system using light graph convolutional network and personalized knowledge-aware attention sub-network. Scientific Reports 2025;**15**:15693.
[19] Li Y, Wang D, Sun Z, Zhang H, Guo H. LightKG: efficient knowledge-aware recommendations with simplified GNN architecture. In: Proceedings of the 31st ACM SIGKDD Conference, Vol. 2; 2025. p. 1577–1588.
[20] Xin H, Sun Y, Wang C, Xiong H. LLMCDSR: enhancing cross-domain sequential recommendation with large language models. ACM Transactions on Information Systems 2025;**43**(5):1–33.
[21] Liu Q, Zhao X, Wang Y, Zhang Z, Zhong H, Chen C, Li X, Huang W, Tian F. Bridge the domains: large language models enhanced cross-domain sequential recommendation. In: Proceedings of the 48th ACM SIGIR Conference; 2025. p. 1582–1592.
[22] Wang S, Zhu J, Wang K, Ruan S. LeCDSR: large language model enhanced cross-domain sequential recommendation. Information Fusion 2026;**127**:103762.
[23] Li Y, Tian T, Cui J, Li D, Fan R, Yan R. Adaptive knowledge-augmented multi-domain recommendation with large language models. Information Fusion 2026;**126**:103619.
[24] TMDb. The Movie Database API documentation; 2026. Available from: https://developer.themoviedb.org/
[25] Internet Archive. Open Library APIs; 2026. Available from: https://openlibrary.org/developers/api
[26] Last.fm. Last.fm API documentation; 2026. Available from: https://www.last.fm/api
[27] Hou Y, Li J, He Z, Yan A, Chen X, McAuley J. Bridging language and items for retrieval and recommendation. arXiv preprint arXiv:2403.03952; 2024.